

2iE Master Analyse Spatiale avec Machine Learning

MODULE 4

Deep Learning Spatial et Google Earth Engine

Dr Rodrique KAFANDO

Enseignant Chercheur, Informatique – Intelligence Artificielle

Paramètre	Valeur
Prérequis	Modules 1–4 pipeline ML/DL spatial maîtrisé
Outils	LangChain · HuggingFace Transformers · SAM (Meta AI) · ChromaDB
Données	Images Sentinel-2 BF · Rapports CH PDF · Cartes IPC prédites (M3)
Fil conducteur	Passer de la prédiction à la génération automatique de rapports

1. Des Modèles Spécialisés aux Modèles de Fondation

Les modules précédents ont construit un pipeline rigoureux : feature engineering géospatial (M2), classification IPC par RF (M3), CNN et LSTM sur images et séries temporelles (M4). Chaque modèle fait une seule chose bien. Le Module 5 introduit une rupture : les modèles de fondation (GPT-4, LLaMA, SAM, CLIP) qui font un nombre surprenant de choses raisonnablement bien, guidés par des instructions en langage naturel sans réentraînement.

À retenir

Un modèle fondationnel n'est pas « meilleur » qu'un RF pour prédire l'IPC.
Il fait autre chose : lire un rapport CH, en extraire les alertes, les croiser avec les prédictions M3, rédiger un bulletin en français dans le même pipeline. Le M5 apprend à orchestrer cette capacité.

Famille	Exemples	Application GeoAI principale
LLM Texte	GPT-4, LLaMA-3, Mistral	Synthèse rapports CH, génération bulletins, Q&A géospatial
VLM Vision+Texte	GPT-4V, LLaVA, InternVL	Description cartes IPC, analyse images satellites
SAM Segmentation	SAM v1/v2 (Meta AI)	Délimitation parcelles, détection zones dégradées
CLIP Encodeur	CLIP, RemoteCLIP	Recherche d'images satellites par texte

LLM géospatial	SatCLIP, GeoCLIP	Représentation géographique pour downstream tasks
----------------	------------------	---

2. SAM Segment Anything Model

SAM (Meta AI, 2023) peut segmenter n'importe quel objet dans n'importe quelle image depuis un simple clic sans avoir été entraîné sur cet objet spécifiquement. Son architecture découple l'encodage de l'image (coûteux, une seule fois) et le décodage des masques (rapide, autant de fois qu'on veut). Pour les images satellites BF : délimitation automatique de parcelles agricoles, zones dégradées, cours d'eau.

Définition

SAM trois composants :

- Image Encoder (ViT-H) : encode l'image entière en vecteurs 256D. Exécuté UNE fois.
- Prompt Encoder : encode les prompts utilisateur (points, boîtes) en embeddings.
- Mask Decoder : combine les deux → 3 masques candidats + scores de confiance.

Ce découplage permet de tester des dizaines de prompts sur la même image encodée sans ré-exécuter le coûteux Image Encoder à chaque fois.

2.1 Installation et segmentation par point

```
# pip install segment-anything
# Poids ViT-H :
https://dl.fbaipublicfiles.com/segment_anything/sam_vit_h_4b8939.pth

from segment_anything import sam_model_registry,
SamPredictor
import numpy as np, rasterio

sam =
sam_model_registry['vit_h'](checkpoint='sam_vit_h_4b8939.pt
h')
sam.eval()
predictor = SamPredictor(sam)

# Préparer une image Sentinel-2 pour SAM (RGB uint8)
def prepare_for_sam(tif_path, bands=(3, 2, 1)):
    with rasterio.open(tif_path) as src:
        rgb = src.read(bands).astype(float).transpose(1, 2,
0)
        p2, p98 = np.percentile(rgb, 2, axis=(0,1)),
np.percentile(rgb, 98, axis=(0,1))
```

```

    rgb = np.clip((rgb - p2) / (p98 - p2 + 1e-8), 0, 1)
    return (rgb * 255).astype(np.uint8)

# Segmenter une parcelle depuis un point GPS
def segment_from_gps(predictor, lon, lat, transform):
    col, row = ~transform * (lon, lat)          # GPS →
    pixel
    input_point = np.array([[int(col), int(row)]])
    input_label = np.array([1])                # 1 =
    l'objet est ICI
    masks, scores, _ = predictor.predict(
        point_coords=input_point,
        point_labels=input_label,
        multimask_output=True                  # 3 masques
    )
    best = scores.argmax()
    surface_m2 = masks[best].sum() * 100        # 10m/pixel →
    100m²
    return masks[best], scores[best], surface_m2

# Usage :
# img_rgb = prepare_for_sam('seno_s2_2023.tif')
# predictor.set_image(img_rgb)    # Encodage unique    ~2s
GPU
# mask, score, surface = segment_from_gps(predictor, -0.05,
14.10, src.transform)
# print(f'Parcelle : {surface/10000:.1f} ha    confiance
{score:.2f}')

```

CAS PRATIQUE 1 : Délimiter les parcelles agricoles en province de Séno

La province de Séno (IPC 4) n'a pas de cadastre agricole numérique. On veut estimer la surface cultivée à partir d'une image Sentinel-2 juillet 2023 en utilisant SAM depuis 10 points GPS collectés lors d'une mission terrain.

Énoncé

1. Pourquoi l'Image Encoder doit-il être exécuté UNE SEULE fois avant les 10 points ?
2. Que signifie multimask_output=True ? Quel masque choisir et comment ?
3. SAM a été entraîné sur des photos naturelles. Quelles limites cela implique-t-il pour les images satellites BF (savane, contours flous, résolution 10m) ?
4. Estimer la surface totale si les 10 masques donnent en moyenne 2.4 ha/parcelle.

```
# Simulation du workflow SAM (sans les vrais poids)
import numpy as np

# Simule 10 segmentations depuis 10 points GPS
np.random.seed(42)
n_points = 10
surfaces_ha = np.random.normal(2.4, 0.6, n_points).clip(0.5, 5.0)
scores_sam = np.random.uniform(0.80, 0.97, n_points)

print('=== Résultats SAM Séno juillet 2023 ===')
for i in range(n_points):
    print(f'Parcelle {i+1:02d}: {surfaces_ha[i]:.2f} ha  
score SAM: {scores_sam[i]:.3f}')

print(f'Total : {surfaces_ha.sum():.1f} ha')
print(f'Moyenne : {surfaces_ha.mean():.2f} ha/parcelle')

# Impact du nombre d'encodages
t_encode = 2.0 # secondes ViT-H sur GPU
t_predict = 0.05 # secondes Mask Decoder
t_1_encode = t_encode + n_points * t_predict
t_10_encode = n_points * (t_encode + t_predict)
print(f'1 encodage + 10 prédictions : {t_1_encode:.1f}s')
print(f'10 encodages + 10 prédictions: {t_10_encode:.1f}s')
print(f'Gain encodage unique : {t_10_encode/t_1_encode:.1f}x  
plus rapide')
```

Sortie attendue

Total : 24.2 ha sur 10 parcelles

1 encodage + 10 prédictions : 2.5s

10 encodages + 10 prédictions : 20.5s

Gain encodage unique : 8.2x plus rapide

Interprétation

L'encodage unique est fondamental : Image Encoder ViT-H = ~2s sur GPU.

Avec 10 prompts : 1 encodage + $10 \times 0.05s = 2.5s$ vs $10 \times 2.05s = 20.5s$ (8x plus rapide).

multimask_output=True retourne 3 masques (petit/moyen/grand) avec scores :

choisir scores.argmax() = le masque le mieux noté par SAM.

Limites sur images satellites BF : parcelles sahéniennes à contours flous,

résolution 10m (un pixel = 100m² petites parcelles invisibles),
pas de données d'entraînement SAM sur savane africaine → biais possible.
SAM v2 (2024) améliore significativement sur les images satellites.

Attention

SAM a été entraîné sur SA-1B (photos naturelles). Ses performances sur images satellites africaines avec végétation éparse n'ont pas été benchmarkées systématiquement. Valider sur un échantillon de parcelles vérifiées terrain avant tout usage opérationnel pour la DGREE ou le CILSS.

3. VLM Description Automatique de Cartes

Les VLM (Vision-Language Models) traitent conjointement image et texte. Contrairement aux CNN qui classifient en une étiquette prédéfinie, un VLM peut décrire une carte en français, répondre à des questions sur son contenu, comparer deux cartes IPC. GPT-4V (OpenAI, payant) et LLaVA (open source, Colab GPU) sont les deux options recommandées pour ce cours.

3.1 Analyser une carte IPC avec GPT-4V

```
import base64, requests

def analyze_ipc_map(image_path, api_key, context=''):
    with open(image_path, 'rb') as f:
        img_b64 =
base64.b64encode(f.read()).decode('utf-8')

    prompt = f'''Tu es expert en sécurité alimentaire pour
l'Afrique de l'Ouest.
Analyse cette carte des phases IPC pour le Burkina
Faso.
Légende : Vert=Phase 1, Jaune=Phase 2, Orange=Phase 3,
Rouge=Phase 4.
{context}
Rédige un bulletin (5-7 lignes) incluant :
1. Zones Phase 3-4 et localisation géographique
2. Zones de résilience (Phase 1-2)
3. Une recommandation pour le CILSS
Style : rapport humanitaire officiel en français.'''

    resp = requests.post(
        'https://api.openai.com/v1/chat/completions',
        headers={'Authorization': f'Bearer {api_key}'},
```

```
        json={
            'model': 'gpt-4-vision-preview',
            'messages': [{ 'role': 'user', 'content': [
                { 'type': 'text', 'text': prompt },
                { 'type': 'image_url', 'image_url': {
                    'url':
f'data:image/png;base64,{img_b64}',
                    'detail': 'high' } }
            ] } ],
            'max_tokens': 500
        }
    )
    return resp.json()[ 'choices' ][ 0 ][ 'message' ][ 'content' ]
```

3.2 LLaVA alternative open source sur Colab GPU

```
from transformers import LlavaNextProcessor,
LlavaNextForConditionalGeneration
from PIL import Image
import torch

# Modèle LLaVA-Next-Mistral 7B tient sur GPU T4 Colab
processor = LlavaNextProcessor.from_pretrained(
    'llava-hf/llava-v1.6-mistral-7b-hf')
model_llava =
LlavaNextForConditionalGeneration.from_pretrained(
    'llava-hf/llava-v1.6-mistral-7b-hf',
    torch_dtype=torch.float16,
    device_map='auto'
)

def describe_map_llava(image_path, question):
    image = Image.open(image_path).convert('RGB')
    prompt = f'[INST] <image>\n{question} [/INST]'
    inputs = processor(prompt, image,
return_tensors='pt').to('cuda')
    with torch.no_grad():
        output = model_llava.generate(
            **inputs, max_new_tokens=300, do_sample=False)
    return processor.decode(output[0],
skip_special_tokens=True)
```

CAS PRATIQUE 2 Générer un bulletin depuis la carte IPC prédite (M3)

La carte des phases IPC prédites par le RF M3 est sauvegardée en PNG. Utiliser un VLM pour en produire automatiquement un bulletin de situation destiné aux coordinateurs OCHA de Ouagadougou.

Énoncé

1. Produire la carte IPC avec matplotlib et la sauvegarder en PNG.
2. Simuler la réponse VLM avec une fonction `describe_ipc_map(df)`.
3. Quelles informations la description VLM ne peut-elle PAS fournir (que contient la carte mais que le VLM ne « voit » pas) ?
4. Rédiger le prompt optimal pour obtenir un bulletin de 5 lignes.

```
import pandas as pd, geopandas as gpd, matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from shapely.geometry import Point

# Données provinces BF (sortie M3)
data = {
    'province':
    ['Séno', 'Soum', 'Oudalan', 'Yatenga', 'Kadiogo', 'Houet', 'Comoé'],
    'ipc_phase': [4, 4, 4, 3, 2, 2, 1],
    'lat': [14.10, 13.98, 14.48, 13.55, 12.36, 11.18, 10.60],
    'lon': [-0.10, -1.55, -0.48, -2.35, -1.53, -4.30, -4.65],
}
df = pd.DataFrame(data)
gdf = gpd.GeoDataFrame(
    df, geometry=[Point(r.lon, r.lat) for _, r in df.iterrows()],
    crs='EPSG:4326')

palette = {1:'#27ae60', 2:'#f1c40f', 3:'#e67e22', 4:'#e74c3c'}
fig, ax = plt.subplots(figsize=(10, 8))
for ipc, grp in gdf.groupby('ipc_phase'):
    grp.plot(ax=ax, color=palette[ipc],
             markersize=200, marker='o', label=f'Phase {ipc}', alpha=0.85)
```

```
for _, row in gdf.iterrows():
    ax.annotate(row['province'], xy=(row.geometry.x,
row.geometry.y),
                xytext=(5, 5), textcoords='offset points',
                fontsize=8)
leg = [mpatches.Patch(color=v, label=f'Phase {k}') for k,v
in palette.items()]
ax.legend(handles=leg, loc='lower left')
ax.set_title('Phases IPC prédites BF (RF M3)')
ax.axis('off')
plt.savefig('carte_ipc_bf.png', dpi=150,
bbox_inches='tight')
plt.show()

# Simulation VLM (remplacer par appel GPT-4V ou LLaVA)
def describe_ipc_map(df):
    p4 = df[df.ipc_phase >= 4].province.tolist()
    p3 = df[df.ipc_phase == 3].province.tolist()
    p1 = df[df.ipc_phase <= 1].province.tolist()
    return (
        f'La carte révèle une situation critique dans le
Nord-Est du BF. '
        f'Provinces Phase 4 (rouge) : {p4}. '
        f'Phase 3 (orange) : {p3}. '
        f'Le gradient N-S est marqué : {p1} (Phase 1) au
Sud-Ouest. '
        f'Recommandation : pré-positionner les stocks à Dori
et Djibo (Séno/Soum). '
    )

bulletin_vlm = describe_ipc_map(df)
print(bulletin_vlm)
```

Sortie attendue

Carte sauvegardée : carte_ipc_bf.png

Bulletin VLM simulé : 5 lignes, provinces Phase 4 identifiées, recommandation CILSS

Interprétation

Ce que le VLM décrit bien : la répartition géographique des couleurs,
le gradient Nord-Sud, les provinces les plus touchées.

Ce que le VLM ne peut PAS fournir depuis l'image seule :

- La valeur numérique exacte du Kappa ($\kappa=0.45$) non visible dans l'image
 - La confiance du modèle par province (probabilités RF)
 - Les données FAPAR sous-jacentes qui expliquent le pattern
 - Le contexte temporel (est-ce une détérioration vs l'année passée ?)
- => Le VLM complète le pipeline M3 il ne le remplace pas.

4. Agent LangChain Requêtes Géospatiales en Langage Naturel

Un LLM seul ne peut pas interroger une base de données, exécuter du code Python, ou accéder aux archives GEE. LangChain permet de construire des agents qui raisonnent (mécanisme ReAct : Reasoning + Acting), choisissent quel outil appelé, l'exécute, et synthétisent les résultats en langage naturel.

Définition

Agent LangChain : un LLM auquel on donne des outils (fonctions Python).

Le cycle ReAct :

1. RÉFLEXION le LLM analyse la question et décide quel outil utiliser.
2. ACTION il appelle l'outil avec les bons arguments.
3. OBSERVATION il lit le résultat de l'outil.
4. RÉPÉTITION si la question n'est pas résolue, il choisit un autre outil.
5. RÉPONSE FINALE il synthétise tout en langage naturel.

4.1 Construction d'un agent géospatial BF

```
from langchain.agents import initialize_agent, Tool,
AgentType
from langchain_community.llms import Ollama # LLM local
gratuit
import pandas as pd

df_ipc = pd.read_csv('M2_feature_matrix_BF.csv')

# Outil 1 : interroger les données géospatiales
def query_ipc_data(question: str) -> str:
    q = question.lower()
    if 'phase 4' in q or 'urgence' in q:
        p4 = df_ipc[df_ipc['ipc_phase'] >=
4]['province'].tolist()
        return f'{len(p4)} provinces Phase 4+ : {p4}'
    elif 'fapar' in q:
        low = df_ipc[df_ipc['fapar_mean'] <
0.10]['province'].tolist()
```

```

        return f'Provinces FAPAR < 0.10 : {low}'
    elif 'enclavé' in q or 'route' in q:
        enc = df_ipc[df_ipc['dist_route_km'] > 150]
        return enc[['province',
'dist_route_km']].to_string(index=False)
    return df_ipc[['province', 'ipc_phase',
'fapar_mean']].to_string(index=False)

# Outil 2 : score de risque composite
def score_risque(province: str) -> str:
    if province not in df_ipc['province'].values:
        return f'{province} non trouvée.'
    r = df_ipc[df_ipc['province'] == province].iloc[0]
    s = r['ipc_phase']/5*0.4 + (1-r['fapar_mean'])*0.3 +
r['dist_route_km']/300*0.3
    s = round(min(s, 1.0), 2)
    niv = 'Critique' if s > 0.7 else ('Élevé' if s > 0.5
else 'Modéré')
    return f'{province} : score={s}    {niv}'

tools = [
    Tool(name='RequeteIPCBF', func=query_ipc_data,
        description='Interroge données IPC/FAPAR/routes
des provinces BF.'),
    Tool(name='ScoreRisque', func=score_risque,
        description='Calcule score de risque composite
pour une province nommée.'),
]

# LLM local Mistral (aucun coût, données confidentielles
protégées)
llm = Ollama(model='mistral', temperature=0)
agent = initialize_agent(tools, llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
verbose=True)

```

CAS PRATIQUE 3 : Identifier les provinces à double urgence IPC + enclavement

Le CILSS doit prioriser les provinces pour une mission d'urgence logistique. L'agent doit combiner deux critères : IPC Phase 4 ET distance aux routes > 150 km. Tester la chaîne de raisonnement ReAct complète.

Énoncé

1. Implémenter les deux outils (RequeteIPCBF + ScoreRisque) avec les fonctions ci-dessus.
2. Lancer la requête : "Quelles provinces sont en Phase 4 ET enclavées ?"
3. Observer le cycle ReAct : combien d'appels d'outils sont nécessaires ?
4. Pourquoi utiliser Mistral local plutôt que GPT-4 pour les données CH confidentielles ?

```
import pandas as pd

# Données simulées (remplacer par M2_feature_matrix_BF.csv
# en production)
df_ipc = pd.DataFrame({
    'province'      :
    ['Séno', 'Soum', 'Oudalan', 'Yatenga', 'Kadiogo', 'Houet'],
    'ipc_phase'     : [4, 4, 4, 3, 2, 2],
    'fapar_mean'    : [0.06, 0.07, 0.06, 0.13, 0.25, 0.38],
    'dist_route_km': [198, 220, 245, 85, 12, 22],
})

# Simulation du cycle ReAct (sans LLM réel)
def simule_react(question):
    print(f'Question : {question}')
    print()
    # Étape 1 : Réflexion
    print('RÉFLEXION : La question mentionne Phase 4 et
    enclavement.')
    print('          Je vais d\'abord chercher les
    provinces Phase 4.')
    # Étape 2 : Action 1
    p4 = df_ipc[df_ipc['ipc_phase'] >=
    4]['province'].tolist()
    print(f'ACTION      : RequeteIPCBF("Phase 4")')
    print(f'RÉSULTAT   : {p4}')
    # Étape 3 : Réflexion 2
    print('RÉFLEXION : Maintenant je filtre sur dist_route >
    150 km.')
    # Étape 4 : Action 2
    enc = df_ipc[df_ipc['dist_route_km'] >
    150]['province'].tolist()
    print(f'ACTION      : RequeteIPCBF("enclavée route")')
    print(f'RÉSULTAT   : {enc}')
    # Étape 5 : Intersection
```

```
double = [p for p in p4 if p in enc]
print(f'RÉFLEXION : Intersection Phase4 et enclavées = {double}')
print()
print(f'RÉPONSE : {len(double)} provinces Phase 4 ET enclavées : {double}.')
print(f'                Distances : {df_ipc[df_ipc.province.isin(double)]["dist_route_km"].tolist() } km')

simule_react('Quelles provinces sont en Phase 4 ET enclavées (dist > 150km) ?')

# Confidentialité : pourquoi Mistral local ?
print()
print('=== Confidentialité des données CH ===')
print('GPT-4 via API OpenAI : les données sont envoyées sur serveurs US.')
print('Mistral via Ollama : traitement 100% local, données ne quittent pas le serveur.')
print('Règle : pour les données sensibles CILSS/CH, toujours utiliser un LLM local.')
```

Sortie attendue

3 provinces Phase 4+ : Séno, Soum, Oudalan

Provinces enclavées (> 150 km) : Séno (198 km), Soum (220 km), Oudalan (245 km)

Double urgence : Séno, Soum, Oudalan toutes les 3 sont à la fois Phase 4 ET enclavées

Interprétation

Le cycle ReAct a nécessité 2 appels d'outils (Phase 4, puis enclavement).

L'agent a raisonné correctement sans qu'on lui ait dit de faire 2 requêtes.

Mistral local est indispensable pour les données CH qui contiennent :

- Données sécuritaires (zones de conflit, mouvements de troupes)
- Statistiques de vulnérabilité par ménage (données personnelles)
- Positions des acteurs humanitaires (sécurité opérationnelle)

Envoyer ces données à l'API OpenAI = violation des accords de confidentialité CH.

5. RAG Interroger les Rapports en Langage Naturel

Un LLM à un knowledge cutoff : il ne connaît pas les rapports CH de 2024. Il peut inventer des statistiques IPC plausibles mais fausses hallucinations. Le RAG (Retrieval-Augmented

Generation) résout ce problème : les rapports réels sont indexés dans une base vectorielle, les passages pertinents sont récupérés à chaque requête, et le LLM génère sa réponse depuis ces passages réels uniquement.

Définition

Pipeline RAG en 4 étapes :

1. Chargement : chaque PDF est découpé en chunks de ~500 tokens.
2. Encodage : chaque chunk est transformé en vecteur (embedding) par un modèle sentence-transformers multilingue (français + anglais).
3. Retrieval : la question est encodée, les k chunks les plus similaires (similarité cosinus) sont récupérés depuis ChromaDB.
4. Génération : le LLM reçoit la question + les k chunks → réponse sourcée.

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import
RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma
from langchain.chains import RetrievalQA

# Étape 1 : Charger et découper les rapports CH
def load_rapports(pdf_paths):
    docs = []
    for path in pdf_paths:
        docs.extend(PyPDFLoader(path).load())
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=500, chunk_overlap=50)
    return splitter.split_documents(docs)

# Étape 2 : Modèle d'embedding multilingue (FR + EN
gratuit, local)
embeddings = HuggingFaceEmbeddings(

model_name='sentence-transformers/paraphrase-multilingual-m
pnet-base-v2'
)

# Étape 3 : Indexation dans ChromaDB
# chunks = load_rapports(['CH_BF_2024_T1.pdf',
'fews_net_BF_2024.pdf'])
# vectordb = Chroma.from_documents(
```

```
#     chunks, embeddings,
persist_directory='./chroma_ch_bf')

# Étape 4 : Pipeline Q&A avec citations
# vectordb = Chroma(persist_directory='./chroma_ch_bf',
#                   embedding_function=embeddings)
# qa = RetrievalQA.from_chain_type(
#     llm=llm_local, chain_type='stuff',
#     retriever=vectordb.as_retriever(search_kwargs={'k':
3})),
#     return_source_documents=True
# )
# result = qa('Quelles provinces BF sont en urgence Phase 4
T1 2024 ?')
# print(result['result'])
# for doc in result['source_documents']:
#     print(f'Source : {doc.metadata["source"]}, page
{doc.metadata["page"]}')

```

CAS PRATIQUE 4 : RAG vs LLM seul le danger des hallucinations

Tester la différence concrète entre un LLM qui répond depuis sa mémoire (risque hallucination) et un LLM ancré par RAG dans les vrais rapports CH. Les données CH BF 2024 sont simulées ici.

Énoncé

1. Implémenter un mini-RAG TF-IDF sur 5 rapports simulés.
2. Poser la même question avec et sans RAG : observer la différence.
3. Identifier les signaux d'hallucination dans la réponse sans RAG.
4. Quelle règle de gouvernance appliquer avant de diffuser un bulletin LLM ?

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Corpus simulé (remplacer par vrais rapports CH en PDF)
corpus = [
    'La province de Séno est en Phase 4. Déficit
pluviométrique 2023 = 35%',
    'Oudalan : 145 000 personnes en urgence alimentaire.
Marchés inactifs.',

```

```
'Les Hauts-Bassins (Houet, Comoé) maintiennent Phase 2.
Excédent +18%.',
'Prévisions sécheresse Sahel BF : probabilité 70% pour
Soum, Séno, Oudalan.',
'Centre-Nord : bascule vers Phase 3. FAPAR juillet 0.14
(-12% vs 2022).',
]
sources = ['CH_T1_2024 p.8', 'CH_T1_2024 p.12', 'FEWS_mars24
p.5',
           'ANAM_2024 p.3', 'FEWS_mars24 p.7']

vect = TfidfVectorizer(ngram_range=(1, 2))
tfidf = vect.fit_transform(corpus)

def rag_query(question, k=2):
    qv = vect.transform([question])
    scores = cosine_similarity(qv, tfidf).flatten()
    top = scores.argsort()[::-1][:k]
    return [(corpus[i], sources[i], round(scores[i], 3)) for
            i in top]

question = 'Situation alimentaire province de Séno 2024 ?'

# Réponse SANS RAG (simulée LLM invente)
sans_rag = ('La province de Séno présente un taux de
malnutrition de 42%'
            ' avec 89 000 personnes affectées selon le
rapport CH T2 2024.'
            ' [ATTENTION : chiffres INVENTÉS par le LLM
non sourcés]')

# Réponse AVEC RAG (ancrée dans les vrais documents)
passages = rag_query(question)
avec_rag = ' | '.join([f'[{s}] {t[:80}]' for t,s,_ in
passages])

print('=== SANS RAG (risque hallucination) ===')
print(sans_rag)
print()
print('=== AVEC RAG (sources citées) ===')
print(avec_rag)
print()
```

```
print('=== Signaux d\'hallucination à détecter ===')
print('1. Chiffres précis non sourcés (42%, 89 000, T2 2024) ')
print('2. Références à des documents inexistants (CH T2 quand seul T1 existe)')
print('3. Contradictions avec les données connues du modèle M3')
print()
print('Règle de gouvernance : tout bulletin LLM doit être validé par')
print('un analyste humanitaire avant diffusion. LLM = premier draft.')
```

Sortie attendue

SANS RAG : chiffres inventés (42%, 89 000, T2 2024 inexistant)

AVEC RAG : passages réels cités avec source et page

Signaux hallucination : précision non sourcée, documents inexistants

Interprétation

Sans RAG, le LLM produit un bulletin convaincant MAIS faux.

Il invente le taux (42% au lieu des ~35% réels), le nombre de personnes (89 000 au lieu de 145 000 pour Oudalan), et cite un rapport inexistant (T2 2024).

Avec RAG : chaque affirmation est ancrée dans un vrai document avec citation.

Règle absolue pour l'usage humanitaire :

1. Toujours utiliser RAG pour les données factuelles
2. Afficher les sources à l'utilisateur final
3. Un expert humain valide et signe avant toute diffusion officielle
4. LLM local pour les données confidentielles CH/CILSS

6. Pipeline GeoAI Complet : Données → Bulletin

Les composants précédents s'assemblent en un pipeline bout-en-bout qui transforme les prédictions ML en bulletins humanitaires actionnables. L'ordre est : prédiction M3 → carte → description VLM → contexte RAG → bulletin LLM. Chaque étape apporte quelque chose que les autres ne peuvent pas fournir.

Étape	Composant	Input	Output
1. Prédiction	RF/XGBoost M3	Feature matrix 45 prov.	ipc_phase prédit
2. Carte	Folium/Matplotlib	GeoDataFrame + phases	Image PNG choroplèthe
3. Segmentation	SAM	Image S2 + zones Phase 4	Masques polygones
4. Description	GPT-4V ou LLaVA	Image carte PNG	Texte, description FR
5. Contexte	RAG (ChromaDB)	Rapports CH PDF + requête	Passages sourcés
6. Bulletin	LLM (Mistral)	Description + contexte	Bulletin alerte FR

```
def generate_bulletin(df_pred, rapports_corpus, llm):
    # 1. Statistiques des prédictions M3
    urgences = df_pred[df_pred['ipc_phase'] >= 4]
    crises = df_pred[df_pred['ipc_phase'] == 3]
    stats = (
        f'{len(urgences)} provinces Phase 4'
        f'({urgences.province.tolist()}), '
        f'{len(crises)} Phase 3.'
    )

    # 2. Description VLM (simulée remplacer par appel GPT-4V)
    vlm_desc = ('Gradient N-S marqué. Sahel en rouge (Phase 4) .'
               ' Sud-Ouest en vert (Phase 1-2).')

    # 3. Contexte RAG
    vect = TfidfVectorizer(); tfidf = vect.fit_transform(rapports_corpus)
    q_vec = vect.transform(['urgence alimentaire Phase 4'])
    scores = cosine_similarity(q_vec, tfidf).flatten()
    top3 = scores.argsort()[::-1][:3]
    context = ' | '.join([rapports_corpus[i][:120] for i in top3])
```

```

# 4. Prompt structuré pour le LLM
prompt = f'''
Tu es analyste humanitaire CILSS. Rédige un bulletin
d\'alerte précoce.
Données ML : {stats}
Analyse carte : {vlm_desc}
Documents CH : {context}
Format : Situation → Zones critiques → Tendances →
Recommandations.
10 lignes max. Français. Style rapport officiel.
Note finale : BULLETIN GÉNÉRÉ AUTOMATIQUEMENT
VALIDATION HUMAINE REQUISE.
'''

# En production : return llm.predict(prompt)
return f'[Bulletin généré depuis prompt de
{len(prompt)} caractères]'

```

7. Limites, Risques et Gouvernance

Le M5 présente des technologies puissantes dont l'usage dans des contextes humanitaires requiert une vigilance particulière. Une décision d'allocation d'aide basée sur un bulletin LLM halluciné peut avoir des conséquences graves sur des populations vulnérables. Cette section est la plus importante du module.

Risque	Manifestation en contexte BF	Mitigation
Hallucination	LLM invente des statistiques IPC ou cite des provinces incorrectes	RAG obligatoire + citations vérifiables
Biais géographique	Données d'entraînement LLM peu représentatives du Sahel africain	Validation par expert terrain avant usage
Confidentialité	Données CH sensibles envoyées à API cloud (OpenAI)	LLM local (Mistral via Ollama) obligatoire
Knowledge cutoff	LLM ignore les rapports CH publiés après son entraînement	Base RAG mise à jour à chaque cycle CH
Confiance excessive	Utilisateur croit le bulletin sans vérifier les sources	Formation + affichage obligatoire des sources

Attention

Un bulletin LLM ne doit JAMAIS être diffusé sans validation humaine.

Il peut servir de premier draft que l'analyste corrige, complète et signe.

Règle de gouvernance : LLM = assistant de rédaction, expert = décideur.

Afficher systématiquement : « Bulletin généré automatiquement validation par [nom de l'analyste] requise avant tout usage opérationnel. »

7.1 Test de validation avant déploiement

```
# Protocole de validation d'un pipeline LLM/RAG avant
déploiement opérationnel

def validate_rag_pipeline(qa_chain, test_questions,
reference_answers):
    '''
        Valide le pipeline RAG sur un ensemble de questions
avec réponses connues.
        Tester sur les 2-3 derniers cycles CH avant
déploiement.
    '''
    n_correct = 0
    for i, (question, reference) in
enumerate(zip(test_questions, reference_answers)):
        result = qa_chain(question)
        generated = result['result']
        sources = [d.metadata.get('source', '?')
                    for d in result.get('source_documents',
[[]])]

        # Vérifications minimales
        has_source = len(sources) > 0
        not_too_long = len(generated) < 1000 # Pas de
réponse interminable
        mentions_key = any(kw in generated.lower()
                           for kw in
reference.lower().split()[:3])

        valid = has_source and not_too_long and
mentions_key
        n_correct += valid
        print(f'Q{i+1}: {"VALIDE" if valid else "ECHEC"}')
        if not valid:
```

```
        print(f'    Source citée : {has_source}')
        print(f'    Longueur OK   : {not_too_long}')
        print(f'    Contenu OK    : {mentions_key}')

    print(f'Score validation :
{n_correct}/{len(test_questions)}')
    print('Déploiement recommandé si score >= 80%')
```